Module 4 -- Arduino Microcontroller (FINAL PROJECT)

Lesson 34: Module 4 Project -- Digital Piano + Display

★ Star-4 Intermediate-Advanced ⏰ 60--75 minutes (longer -- it is a project!)

👤 Age 6--12 years ⚡ **500 XP (the biggest XP haul of the module!)**

Your Mission Today

This is it, Circuit Explorer — the GRAND FINALE of Module 4! You are going to build a real, working **Digital Piano** that combines EVERYTHING you have learned: digital outputs (LEDs), digital inputs (buttons), analog concepts (PWM), sound (buzzer and tones), display (LCD), arrays, loops, and variables. When you press a button, a musical note plays, the matching LED lights up, and the LCD shows the note name. This is a real instrument you built and programmed yourself. Let us make it happen!

Learning Objectives

This project brings together everything from Module 4:

- Digital outputs: LEDs controlled by code
- Digital inputs: buttons read with digitalRead()
- Sound: piezo buzzer with tone() function
- Display: 16x2 LCD showing note information
- Programming: arrays, for loops, if statements, variables
- Wiring: complex multi-component circuit on breadboard

What You Need

Item	Qty
Arduino Uno + USB cable	1
Breadboard (830-point recommended)	1
Push buttons (tactile switches)	8
LEDs (assorted colors)	8
330-ohm resistors	8
Piezo buzzer (passive)	1
16x2 LCD with I2C backpack	1
Jumper wires	25+
Multimeter (Magic Measurement Wand!)	1

How to Build It

Phase 1: Plan the Piano Layout (5 min)

The 8 Notes (One Octave of C Major):

Button	Note	Frequency	LED Color (suggested)
1	C4	262 Hz	Red
2	D4	294 Hz	Orange (or Red)
3	E4	330 Hz	Yellow
4	F4	349 Hz	Green
5	G4	392 Hz	Green
6	A4	440 Hz	Blue
7	B4	494 Hz	Blue
8	C5	523 Hz	White

💬

> "You are building a full octave — from low C to high C. That is all the white keys on a piano between two C notes!"

Award: **+20 XP** **for planning the piano layout!**

Phase 2: Wire the Buttons (15 min)

Button Wiring (using INPUT_PULLUP — no external resistors needed):

```
Arduino Pin   -->   Button   -->   GND

Pin 2  ---- [BUTTON 1] ---- GND    (C4)
Pin 3  ---- [BUTTON 2] ---- GND    (D4)
Pin 4  ---- [BUTTON 3] ---- GND    (E4)
Pin 5  ---- [BUTTON 4] ---- GND    (F4)
Pin 6  ---- [BUTTON 5] ---- GND    (G4)
Pin 7  ---- [BUTTON 6] ---- GND    (A4)
Pin 12 ---- [BUTTON 7] ---- GND    (B4)
Pin 13 ---- [BUTTON 8] ---- GND    (C5)
```

Steps:

1. Place 8 buttons in a row across the center gap of the breadboard
2. Connect one side of each button to its Arduino pin (2, 3, 4, 5, 6, 7, 12, 13)
3. Connect the other side of ALL buttons to the GND rail
4. Run one wire from the GND rail to Arduino GND

💬

> "We use INPUT_PULLUP mode in the code, so we do not need external pull-up resistors. The Arduino handles it internally. Nice and simple!"

Test Each Button:

Upload a quick test sketch to make sure all buttons work:

```
void setup() {
  Serial.begin(9600);
  for (int pin = 2; pin <= 7; pin++) {
    pinMode(pin, INPUT_PULLUP);
  }
  pinMode(12, INPUT_PULLUP);
  pinMode(13, INPUT_PULLUP);
  Serial.println("Button test ready!");
}

void loop() {
  for (int pin = 2; pin <= 7; pin++) {
    if (digitalRead(pin) == LOW) {
      Serial.print("Button on pin ");
      Serial.print(pin);
      Serial.println(" PRESSED");
    }
  }
  if (digitalRead(12) == LOW) Serial.println("Button on pin 12 PRESSED");
  if (digitalRead(13) == LOW) Serial.println("Button on pin 13 PRESSED");
  delay(100);
}
```

Award: **+40 XP** **for wiring and testing all 8 buttons!**

Phase 3: Wire the Buzzer (5 min)

```
  Arduino Pin 8 ---- Buzzer (+) ---- Buzzer (-) ---- GND
```

Connect the buzzer to **pin 8** (same as Lesson 32).

Award: **+10 XP** **for wiring the buzzer!**

Phase 4: Wire the LCD (5 min)

```
Arduino 5V  ---- LCD VCC
Arduino GND ---- LCD GND
Arduino A4  ---- LCD SDA
Arduino A5  ---- LCD SCL
```

Connect the LCD (same as Lesson 33).

Award: **+10 XP** **for wiring the LCD!**

Phase 5: Wire the LEDs (Optional but Awesome) (10 min)

LED Wiring using PWM and digital pins:

Since we have used pins 2-8, 12, 13, A4, and A5, we can use pins 9, 10, 11 for LEDs and share some with the buttons for a simpler build. However, for a full 8-LED setup, we can use a shift register or reduce to 3 indicator LEDs.

Simplified 3-LED Indicator Version:

```
Pin 9  (~) ---- [330 ohm] ---- LED (Red)   ---- GND   (low notes C,D,E)
Pin 10 (~) ---- [330 ohm] ---- LED (Green) ---- GND   (mid notes F,G)
Pin 11 (~) ---- [330 ohm] ---- LED (Blue)  ---- GND   (high notes A,B,C5)
```

💬

> "Three LEDs show which range you are playing in — red for low notes, green for middle, blue for high. It is like a visual equalizer!"

Award: **+20 XP** **for wiring the indicator LEDs!**

Phase 6: The Piano Code (15 min)

The Complete Digital Piano Sketch:

```
// Lesson 34: Digital Piano with LCD Display
// Module 4 Final Project

#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

// Button pins
int buttonPins[] = {2, 3, 4, 5, 6, 7, 12, 13};
int numButtons = 8;

// Note frequencies (C4 to C5)
int noteFreqs[] = {262, 294, 330, 349, 392, 440, 494, 523};

// Note names for the LCD
String noteNames[] = {"C4", "D4", "E4", "F4", "G4", "A4", "B4", "C5"};

// LED pins (3 indicator LEDs)
int ledLow = 9;    // Red -- C4, D4, E4
int ledMid = 10;   // Green -- F4, G4
int ledHigh = 11;  // Blue -- A4, B4, C5

// Buzzer pin
int buzzerPin = 8;

// Track what is playing
bool notePlaying = false;

void setup() {
  // Set up buttons
  for (int i = 0; i < numButtons; i++) {
    pinMode(buttonPins[i], INPUT_PULLUP);
  }

  // Set up LEDs
  pinMode(ledLow, OUTPUT);
  pinMode(ledMid, OUTPUT);
  pinMode(ledHigh, OUTPUT);

  // Set up LCD
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Digital Piano!");
  lcd.setCursor(0, 1);
```

```
  Serial.begin(9600);
  Serial.println("=== Digital Piano Ready! ===");
}

void loop() {
  bool anyPressed = false;

  for (int i = 0; i < numButtons; i++) {
    if (digitalRead(buttonPins[i]) == LOW) {
      anyPressed = true;

      // Play the note
      tone(buzzerPin, noteFreqs[i]);

      // Light the correct LED
      digitalWrite(ledLow, LOW);
      digitalWrite(ledMid, LOW);
      digitalWrite(ledHigh, LOW);

      if (i < 3) {
        digitalWrite(ledLow, HIGH);   // Low notes: C, D, E
      } else if (i < 5) {
        digitalWrite(ledMid, HIGH);   // Mid notes: F, G
      } else {
        digitalWrite(ledHigh, HIGH);  // High notes: A, B, C5
      }

      // Display on LCD
      lcd.setCursor(0, 0);
      lcd.print("Now Playing:    ");
      lcd.setCursor(0, 1);
      lcd.print("Note: ");
      lcd.print(noteNames[i]);
      lcd.print("  ");
      lcd.print(noteFreqs[i]);
      lcd.print(" Hz  ");

      // Print to Serial Monitor too
      Serial.print("Playing: ");
      Serial.print(noteNames[i]);
      Serial.print(" (");
      Serial.print(noteFreqs[i]);
      Serial.println(" Hz)");
```

```
    }
  }

  if (!anyPressed && notePlaying) {
    noTone(buzzerPin);
    digitalWrite(ledLow, LOW);
    digitalWrite(ledMid, LOW);
    digitalWrite(ledHigh, LOW);

    lcd.setCursor(0, 0);
    lcd.print("Digital Piano!  ");
    lcd.setCursor(0, 1);
    lcd.print("Press a button! ");

    notePlaying = false;
  }

  delay(10);  // Small delay for stability
}
```

Upload and play!

> "Press each button — you hear a different note, the LCD shows which note it is and its frequency, and the LEDs glow in the matching color range. You just built a real musical instrument with a computer brain!"

Award: **+60 XP** **for getting the full piano working!**

Phase 7: Wand-Assisted Debugging (5 min)

> "If something is not working, do not panic! Grab your Magic Measurement Wand and become a circuit detective."

Debugging Checklist:

1. **Power check:** Measure 5V pin to GND. Should read ~5V
2. **Button check:** Measure each button pin to GND. Should read ~5V when NOT pressed, ~0V when pressed
3. **Buzzer check:** Measure pin 8 while pressing a button. Should show ~2.5V average (tone playing)

5. **LCD check:** Measure VCC to GND on the LCD. Should read ~5V

```
| Check Point         | Expected   | Your Reading | OK? |
|--------------------|-----------|-------------|-----|
| Arduino 5V          | ~5.0V      |             |     |
| Button pin (idle)   | ~5.0V      |             |     |
| Button pin (pressed)| ~0.0V      |             |     |
| Buzzer pin (playing)| ~2.5V      |             |     |
| LED pin (active)    | ~5.0V      |             |     |
| LCD VCC             | ~5.0V      |             |     |
```

Award: **+40 XP** **for successfully debugging with the Wand!**

Phase 8: Bonus Challenges (Optional)

Challenge 1 — Play a Song Automatically:

Add a function that plays a pre-programmed song when no buttons are pressed for 10 seconds.

Challenge 2 — Record and Playback:

Record which buttons the player presses (store in an array), then play it back.

Challenge 3 — Two-Octave Piano:

Use the potentiometer to switch between two octaves (C3-C4 and C4-C5).

Award: **+30 XP** **for completing any bonus challenge!**

Module 4 Final Assessment

After building the piano, have the kid answer these (no looking at notes!):

Concept Check:

1. "What is a microcontroller?"
2. "What is the difference between setup() and loop()?"
3. "What does digitalRead() return?"

5. "What does PWM stand for, and what does it do?"
6. "What is an array?"

Code Challenge — Write from Scratch:

"Write a sketch from scratch that blinks an LED on pin 7 five times, then stops."

```
// Expected answer (roughly):
int ledPin = 7;

void setup() {
  pinMode(ledPin, OUTPUT);
  for (int i = 0; i < 5; i++) {
    digitalWrite(ledPin, HIGH);
    delay(500);
    digitalWrite(ledPin, LOW);
    delay(500);
  }
}

void loop() {
  // Empty -- we only want 5 blinks
}
```

Wand Skills Check:

"Measure the 5V pin on the Arduino." "Measure a button pin while pressing and releasing it." "What voltage does the Wand show on a PWM pin set to 127?"

Award: **+50 XP** **for completing the final assessment!**

Module 4 Complete — Grand Finale!

```
=====================================================

   MODULE 4 CHAMPION BADGE UNLOCKED!

   ______________________ has mastered
      Arduino Microcontroller

   [check] What is a Microcontroller
   [check] First Sketch -- Blink
   [check] Variables and Serial Monitor
   [check] Reading Digital Inputs -- Buttons
   [check] Analog Inputs
   [check] PWM -- Dimming LEDs
   [check] For Loops and Arrays
   [check] Piezo Buzzer and Tones
   [check] LCD Display
   [check] Built the Digital Piano!

=====================================================
```

Lesson 34 XP Breakdown:

Activity	XP
Plan piano layout	20
Wire and test 8 buttons	40
Wire buzzer	10
Wire LCD	10
Wire indicator LEDs	20
Complete piano code working	60
Wand debugging	40
Bonus challenges	30
Final assessment	50
TOTAL POSSIBLE	**500** (project bonus!)

Module 4 Total XP Summary

Lesson	Title	Max XP
25	What is a Microcontroller?	250
26	Your First Sketch — Blink	250
27	Variables and Serial Monitor	250
28	Reading Digital Inputs — Buttons	250
29	Analog Inputs	250
30	PWM — Dimming LEDs	250
31	For Loops and Arrays	250
32	Piezo Buzzer and Tones	250
33	LCD Display (16x2)	250
34	Digital Piano Project	500
MODULE TOTAL		**2,750**

XP Ranks:

- 0—600 XP: Arduino Apprentice
- 601—1,200 XP: Arduino Explorer
- 1,201—1,800 XP: Arduino Builder
- 1,801—2,400 XP: Arduino Engineer
- 2,401—2,750 XP: Arduino Champion!

All Badges Earned in Module 4

Badge	Lesson
Microcontroller Explorer	25
Blink Master	26
Variable Wizard	27
Input Handler	28
Analog Adventurer	29
PWM Wizard	30
Loop Legend	31
Melody Maker	32
Display Commander	33
Module 4 Champion	34

Reflection Questions (Talk Through Together)

💬

"What was the coolest thing you programmed in this module?" "What was the hardest concept to understand?" "Which project would you want to build next using Arduino?" "If you could add one more feature to your piano, what would it be?" "How do you think sensors (like temperature or distance) could make the piano even cooler?"

What's Next — Module 5 Preview

💬

> "Now you know how to program an Arduino and control LEDs, buttons, buzzers, and displays! In Module 5, we are going to connect SENSORS — devices that let the Arduino see, feel, and measure the real world. Temperature sensors, light sensors, distance sensors, motion detectors, and MOTORS! You will build gadgets that react to their environment automatically. Get ready for Sensors and Actuators!"

Parent/Instructor Notes

- This project takes longer than a regular lesson — plan for 60—75 minutes or split across two sessions.
- **Session 1:** Wire buttons, buzzer, and test. **Session 2:** Add LCD, LEDs, and final code.
- The most common issue is wiring mistakes with 8 buttons. Encourage the child to test each button individually before writing the piano code.
- The Wand debugging phase is extremely valuable. Let the child lead the debugging process.
- If you run out of pins, the 3-LED indicator approach (low/mid/high) works great. A full 8-LED version requires a shift register (74HC595), which is a great extension project.
- Encourage the child to perform a mini concert for family members and explain how each part works.
- The Digital Piano is a fantastic project to photograph or video for a portfolio.
- Consider having the child teach a younger sibling how to play a simple song on their piano.